COMP0008:
Computer Architecture and Concurrency

Live Session (week 9)

Stefano Vissicchio

References: GPBBHL chapters 3, 4 and 14

Warmup

- Self-assess on www.menti.com w/ code 7463 0735
  - would help me try and tune review speed

- Interactions have been great so far
  - let's keep going!

- Successful experiments: will use Mentimeter polls *during* the session
  - for feedback on my pace and in-class questions
  - my understanding is that you like them! :)

The concurrency story so far

- OSes run threads concurrently
  - possibly on different processors
- Interference: threads may update shared data at the same time, and hence inconsistently
  - this can lead to incorrect object's state (e.g., mixed up servlet cache entries) and behaviour (e.g., returning the output for a different input)
- To ensure correctness, programmers ***must*** prevent multiple threads from simultaneously accessing the same shared data
  - basic mechanism in Java: synchronization with (implicit) locking, to enforce mutual exclusion for the execution of critical sections of the code

Let's check this example: intention

```
public class Example {
  private static boolean ready;
  private static int number;
  private static class Reader extends Thread {
    public void run() {
      while(!ready)
        Thread.sleep(1000);
      System.out.println(number);
    }
  }
  public static void main(String[] args){
    new Reader().start();
    number = 42;
    ready = true;
  }
}
```

A Reader thread spins until ready is true: when this happens, it simply prints the value of the number field.

Main thread creates a Reader thread, sets number to 42, and then allows Reader to print (and exit). **Expectation**: 42 is printed.

Let's check this example: isn't it thread safe?

```
public class Example {
  private static boolean ready;
  private static int number;
  private static class Reader extends Thread {
    public void run() {
      while(!ready)
        Thread.sleep(1000);
      System.out.println(number);
    }
  }
  public static void main(String[] args){
    new Reader().start();
    number = 42;
    ready = true;
  }
}
```

shared among Main and Reader threads

The Reader thread *only reads* the shared variables: the two threads do **not interfere**, nor induce spurious state changes

Let's check this example: can loop forever

```
public class Example {
  private static boolean ready;
  private static int number;
  private static class Reader extends Thread {
    public void run() {
      while(!ready)
        Thread.sleep(1000);
      System.out.println(number);
    }
  }
  public static void main(String[] args){
    new Reader().start();
    number = 42;
    ready = true;
  }
}
```

No guarantee that state changes are propagated to other threads!!

Consequence: Reader may **never** see ready set to true

Let's check this example: zero may be printed

```
public class Example {
  private static boolean ready;
  private static int number;
  private static class Reader extends Thread {
    public void run() {
      while(!ready)
        Thread.sleep(1000);
      System.out.println(number);
    }
  }
  public static void main(String[] args){
    new Reader().start();
    number = 42;
    ready = true;
  }
}
```

No guarantee that consecutive assignments are executed in order, nor that all caches are coherent at any moment in time!

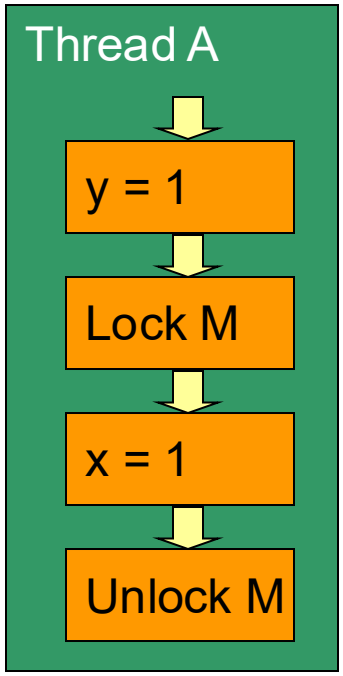
What about the concurrency abstraction?

- We defined interleaving by constraining actions within each thread to be executed in order
  - because hardware guarantees per-thread correctness
- However, we've just seen that actions of one thread can be seen by other threads as reordered
  - when reordering does not affect per-thread correctness
- Observations about the concurrency abstraction
  - It is a model, hence it abstracts from reality (as all models)
  - It is still useful for interference problems
    - e.g., if we are sure that there are no other problems
  - *Sometimes*, it may be extended to account for reordering by modeling independent instructions as different "threads"
    - in last example, T1:{number=42} and T2:{ready=true}

These are *visibility* problems

- Problem: one thread changes the state, but the other thread does not see the (entire) new state
  - Reader sees a partial change when it prints zero
  - Reader sees no change when it doesn't terminate
- Analysis: this is different from the interference problems we've seen last week
  - It is rather the opposite: interference is about "too much interaction" between threads, visibility about "too little"
- Solution: we need (some form of) synchronization whenever one thread accesses data shared with other threads
  - i.e., *both reads and writes* must be synchronized

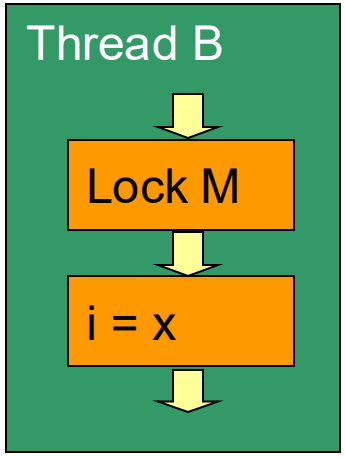
Why synchronization?



Java specification states that all threads must "*synchronize their working memories*" with the main memory at the *entry and exit to synchronized* segments.

Everything before the unlock of **M** within Thread A…

… is visible to everything after the lock of **M** within Thread B



A different example

```
public class ConfigSettings {
  public Holder holder;

  public ConfigSettings() {
    holder = new Holder(42);
  }
}

public class Holder {
  private int n;

  public Holder(int n) {this.n = n;}

  public void MyTest() {
    if (n != n)
      throw new AssertionError(“error!”);
  }
}
```

A different example

```
public class ConfigSettings {
  public Holder holder;

  public ConfigSettings() {
    holder = new Holder(42);
  }
}

public class Holder {
  private int n;

  public Holder(int n) {this.n = n;}

  public void MyTest() {
    if (n != n)
      throw new AssertionError("error!");
  }
}
```

Two threads A and B:
[A] calls new Holder(42)
[B] gets a ref to holder
[B] calls MyTest()
[B] reads n=0 in MyTest()
[A] sets this.n=42
[B] reads n=42 in MyTest

Outcome: AssertionError is raised because B sees *a partially constructed* Holder object!

A different example

```
public class ConfigSettings {
  public Holder holder;

  public ConfigSettings() {
    holder = new Holder(42);
  }
}

public class Holder {
  private int n;

  public Holder(int n) {this.n = n;}

  public void MyTest() {
    if (n != n)
      throw new AssertionError("error!");
  }
}
```

Atomic actions: two reads in myTest(), plus several actions in the constructor (e.g., instantiating + initializing fields + publish reference to the new object)

Problem: if constructor and myTest() are run by different threads, the atomic actions can be interleaved arbitrarily

Avoiding visibility problems

- For any given shared object, we have *two* high-level goals, really:
  - G1: no thread sees a partially constructed object
  - G2: all threads see changes to the object's state

- To achieve those goals, we need to understand the Java Memory Model (JMM)
  - The JMM spells out the guarantees that the JVM provides about visibility of one thread's writes to other threads
  - *note*: synchronization prevents any instruction reordering that would violate JMM guarantees

- Publication and safe publication stem from the JMM

What does publication mean?

- An object is published when it is made available to code outside its current context
  - e.g., storing it in a public field, returning it from a public method, passing it as parameter, ...
- Hazard: publishing an object in a constructor can make a partially constructed object available to external code (e.g., other threads)

```
public class ConfigSettings {
  public Holder holder;

  public ConfigSettings() {
    holder = new Holder(42);
  }
}
```

since the holder field is public, the Holder object is published here!

What does safe publication mean?

- An object is published when it is made available to code outside its current context
  - e.g., storing it in a public field, returning it from a public method, passing it as parameter, ...
- Hazard: publishing an object in a constructor can make a partially constructed object available to external code (e.g., other threads)
- An object is safely published if both the reference to the object and its state are made visible to other threads at the same time
  - We will first discuss when safe publication is needed, and later revise safe publication idioms in Java

Avoiding visibility problems

- For any given shared object, we have *two* high-level goals, really:
  - G1: no thread sees a partially constructed object
  - G2: all threads see changes to the object's state
- Achieving those goals depends on the object's type

Object type	Goal G1	Goal G2
Local to thread		

not shared

Avoiding visibility problems

- For any given shared object, we have *two* high-level goals, really:
  - G1: no thread sees a partially constructed object
  - G2: all threads see changes to the object's state
- Achieving those goals depends on the object's type

Object type	Goal G1	Goal G2
Local to thread	✓	✓

Avoiding visibility problems

- For any given shared object, we have *two* high-level goals, really:
  - G1: no thread sees a partially constructed object
  - G2: all threads see changes to the object's state
- Achieving those goals depends on the object's type

Object type	Goal G1	Goal G2
Local to thread	✓	✓
Immutable		

unmodifiable state,
all fields are final,
properly constructed

Avoiding visibility problems

- For any given shared object, we have *two* high-level goals, really:
  - G1: no thread sees a partially constructed object
  - G2: all threads see changes to the object's state
- Achieving those goals depends on the object's type

Object type	Goal G1	Goal G2
Local to thread	✓	✓
Immutable	✓	✓

Avoiding visibility problems

- For any given shared object, we have *two* high-level goals, really:
  - G1: no thread sees a partially constructed object
  - G2: all threads see changes to the object's state
- Achieving those goals depends on the object's type

Object type	Goal G1	Goal G2
Local to thread	✓	✓
Immutable		✓
Effectively immutable		

state doesn't change after publication

Avoiding visibility problems

- For any given shared object, we have *two* high-level goals, really:
  - G1: no thread sees a partially constructed object
  - G2: all threads see changes to the object's state
- Achieving those goals depends on the object's type

Object type	Goal G1	Goal G2
Local to thread	✓	✓
Immutable	✓	✓
Effectively immutable	must be safely published	✓

Avoiding visibility problems

- For any given shared object, we have *two* high-level goals, really:
  - G1: no thread sees a partially constructed object
  - G2: all threads see changes to the object's state
- Achieving those goals depends on the object's type

Object type	Goal G1	Goal G2
Local to thread	✓	✓
Immutable	✓	✓
Effectively immutable		✓
Mutable		

state changes over time, and multiple threads may read or write it

Avoiding visibility problems

- For any given shared object, we have *two* high-level goals, really:
  - G1: no thread sees a partially constructed object
  - G2: all threads see changes to the object's state
- Achieving those goals depends on the object's type

Object type	Goal G1	Goal G2
Local to thread	✓	✓
Immutable	✓	✓
Effectively immutable	must be safely published	✓
Mutable	must be safely published	must be thread-safe or guarded by lock

How can we safely publish an object?

- An object is *safely published* if both the reference to it and its state are made visible to other threads at the same time

- The Java Memory Model guarantees that any of the following idioms is sufficient for safely publishing
  - Initialize object reference from a static initializer
  - Store a reference to it in a volatile field or AtomicReference
  - Store a reference to it into a final field of a properly constructed object
  - Store a reference to it into a field that is properly guarded by a lock

Let's fix the previous example

```
public class ConfigSettings {
  public Holder holder;

  public ConfigSettings() {
    holder = new Holder(42);
  }
}

public class Holder {
  private int n;

  public Holder(int n) {this.n = n;}

  public void MyTest() {
    if (n != n)
      throw new AssertionError("error!");
  }
}
```

the Holder object is **not** safely published

Holder objects are effectively immutable

Option 1: make Holder immutable

```
public class ConfigSettings {
  public Holder holder;

  public ConfigSettings() {
    holder = new Holder(42);
  }
}

public class Holder {
  private final int n;

  public Holder(int n) {this.n = n;}

  public void MyTest() {
    if (n != n)
      throw new AssertionError("error!");
  }
}
```

note: the value of n cannot be changed afterwards

Option 2a: safely publish Holder

```
public class ConfigSettings {
  public static Holder holder = new Holder(42);
}

public class Holder {
  private int n;

  public Holder(int n) {this.n = n;}

  public void MyTest() {
    if (n != n)
      throw new AssertionError(“error!”);
  }
}
```

note: holder becomes a static property of ConfigSettings

Option 2b: safely publish Holder

```
public class ConfigSettings {
  public volatile Holder holder;

  public ConfigSettings() {
    holder = new Holder(42);
  }
}

public class Holder {
  private int n;

  public Holder(int n) {this.n = n;}

  public void MyTest() {
    if (n != n)
      throw new AssertionError(“error!”);
  }
}
```

note: volatile ensures that reads always return the most recent write by any thread (weak synchronization)

Option 2c: safely publish Holder

```
public class ConfigSettings {
  public final Holder holder;

  public ConfigSettings() {
    holder = new Holder(42);
  }
}

public class Holder {
  private int n;

  public Holder(int n) {this.n = n;}

  public void MyTest() {
    if (n != n)
      throw new AssertionError(“error!”);
  }
}
```

note: the reference to the Holder object cannot be changed afterwards

Option 2d: safely publish Holder

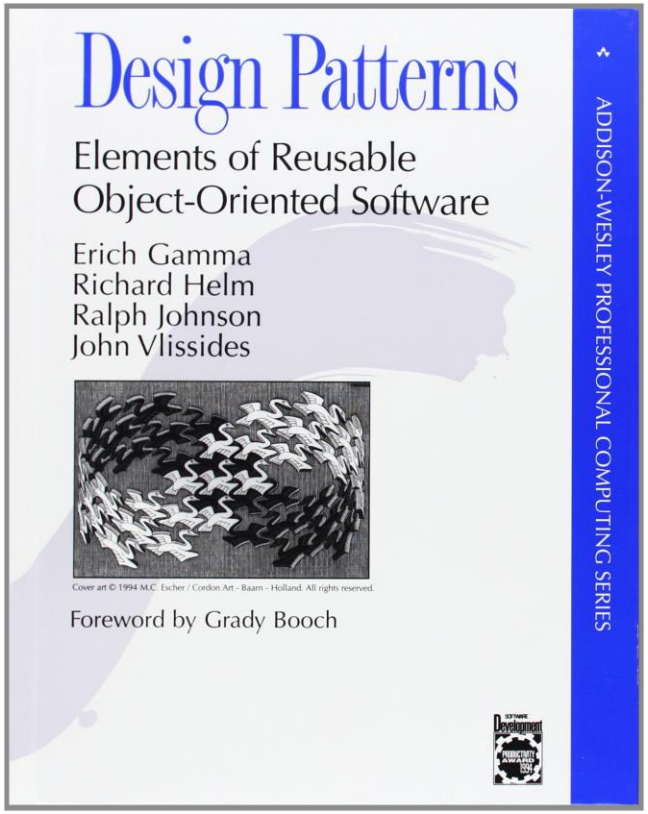
```
public class ConfigSettings {
  private Holder holder;

  public ConfigSettings() {
    holder = new Holder(42);
  }

  public synchronized Holder GetHolder() {
    return holder;
  }
}

public class Holder {
  private int n;
  ...
}
```

note: locking has an impact on performance and interface

Towards large applications: patterns

- Reasoning about interference and visibility problems for each object/state is complex and error-prone
  - yet, correctness must be ensured in all concurrent applications

- We need design patterns!
- Design patterns are general, repeatable solutions to common problems in software engineering
  - encode good practices, built from experience



Towards large applications: monitor pattern

- A classic pattern used to build large concurrent applications in Java is the monitor pattern

- Java monitor pattern: to make an object thread-safe, encapsulate all its mutable state and guard the state with the object's intrinsic lock
  - i.e., all methods that access the state are `synchronized`
  - note the possible simplicity-performance tradeoff

- Ways to *use* the pattern:
  - wrap non-thread-safe objects
  - model data as classes implementing the monitor pattern: they can be safely accessed by multiple threads

Example: a mutable and safe Holder

```
public class MutableHolder {
  private int n = 0;

  public synchronized void SetValue(int x) {
     n = x;
  }

  public synchronized int GetValue() {
    return n;
  }

  public synchronized void MyTest() {
    if (n != n)
      throw new AssertionError(“error!”);
  }
}
```

Towards large applications: conditional sync

- Problem: what if a thread cannot work on the current state of a monitor (e.g., read empty buffer)?
  - fail / raise an error ← only possibility for single-thread apps
  - wait for another thread to change the monitor's state
- Conditional sync supports the second possibility

Towards large applications: conditional sync

- Problem: what if a thread cannot work on the current state of a monitor (e.g., read empty buffer)?
  - fail / raise an error ← only possibility for single-thread apps
  - wait for another thread to change the monitor's state
- Conditional sync supports the second possibility

PSEUDO-CODE

```
acquire lock
while (predicate) {
  release lock
  wait to be notified
  reacquire lock
}
perform action
notify other threads
release lock
```

Towards large applications: conditional sync

- Problem: what if a thread cannot work on the current state of a monitor (e.g., read empty buffer)?
  - fail / raise an error ← only possibility for single-thread apps
  - wait for another thread to change the monitor's state
- Conditional sync supports the second possibility

PSEUDO-CODE

```
acquire lock
while (predicate) {
  release lock
  wait to be notified
  reacquire lock
}
perform action
notify other threads
release lock
```

EXAMPLE

```
public synchronized V read(){
  while(isEmpty()){
    wait();
  }
  V val = doRead();
  notifyAll();
  return val;
}
```

Understanding conditional sync

- Mechanism: condition queues give threads a way to subscribe to specific updates on a given condition
  - waiting threads form a so-called wait set
- Condition queues offer methods to
  - wait() allows threads to enter a condition queue
  - notify() wakes up one thread in the wait set
  - notifyAll() wakes up *all* threads in the wait set
- Important: notifications signal that "something has changed", ***not what*** has changed
  - threads must re-check condition when woken up

Understanding conditional sync

- Each object behaves as a condition queue
  - this.wait() makes a thread wait until this.notifyAll() is called
- Which lock and conditional queue to use?
  - the condition predicate is a condition on state variables
  - before testing the condition predicate, we must hold the lock guarding the corresponding state variables
    - we must also hold that lock when calling wait and notification methods on the condition queue
  - additionally, lock object == condition queue object
- NB: we should think of entry and exit protocols

Additional content

- Additional examples of monitors and conditional synchronization [Moodle]
- Proper use of conditional sync [GPBBHL 14.2]
- Volatile keyword [GPBBHL 3.1.4]
  - semantics (updates to a volatile variable are propagated predictably to other threads) and how to use them

- Additional guidelines for thread safety [GPBBHL 4]
- The actual Java Memory Model [GPBBHL 16]
  - low-level guarantees, and why they translate into the discussed safely publishing rules